WEB & APP DEVELOPMENT

Node.js

Master the Node.js runtime from the event loop to a production-ready Express REST API — the async model, modules, npm, and error handling an engineer and interviewer both expect.

nodejs · event-loop · express · async-await · rest-api · npm · backend

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What Node.js Is and Why It Matters
2. 2. Setup and Getting Started
3. 3. Modules: CommonJS vs ESM
4. 4. The Event Loop in Depth
5. 5. Async Patterns: Callbacks, Promises, async/await
6. 6. npm and the Package Ecosystem
7. 7. Core Modules: Files, Paths, Process, Env
8. 8. Building an HTTP Server
9. 9. Express: Routing, Middleware, and REST
10. 10. Error Handling
11. 11. Worked Mini-Example: A REST API Backend
12. 12. Cheat-Sheet Reference
13. 13. Best Practices and Common Pitfalls
14. 14. Interview Questions & Answers
15. 15. Further Learning

Node.js is a server-side JavaScript runtime built on Chrome's V8 engine. Its single-threaded, non-blocking, event-driven architecture lets one process juggle tens of thousands of concurrent connections — making it the default choice for APIs, real-time services, tooling, and the glue of modern web stacks. This guide takes you from the event loop up to a working Express REST backend.

1. What Node.js Is and Why It Matters

Node.js wraps Google's **V8** JavaScript engine with a C/C++ layer (**libuv**) that provides an event loop, a thread pool, and asynchronous I/O. The result: JavaScript that runs outside the browser, with full access to files, networks, and the operating system.

The defining idea is **non-blocking I/O**. Instead of dedicating one OS thread per request (the classic thread-per-connection model), Node runs your JavaScript on a *single main thread* and offloads slow operations — disk reads, DB queries, network calls — to the kernel or a background thread pool. When they finish, a *callback* is queued back onto the main thread. The thread is almost never idle waiting.

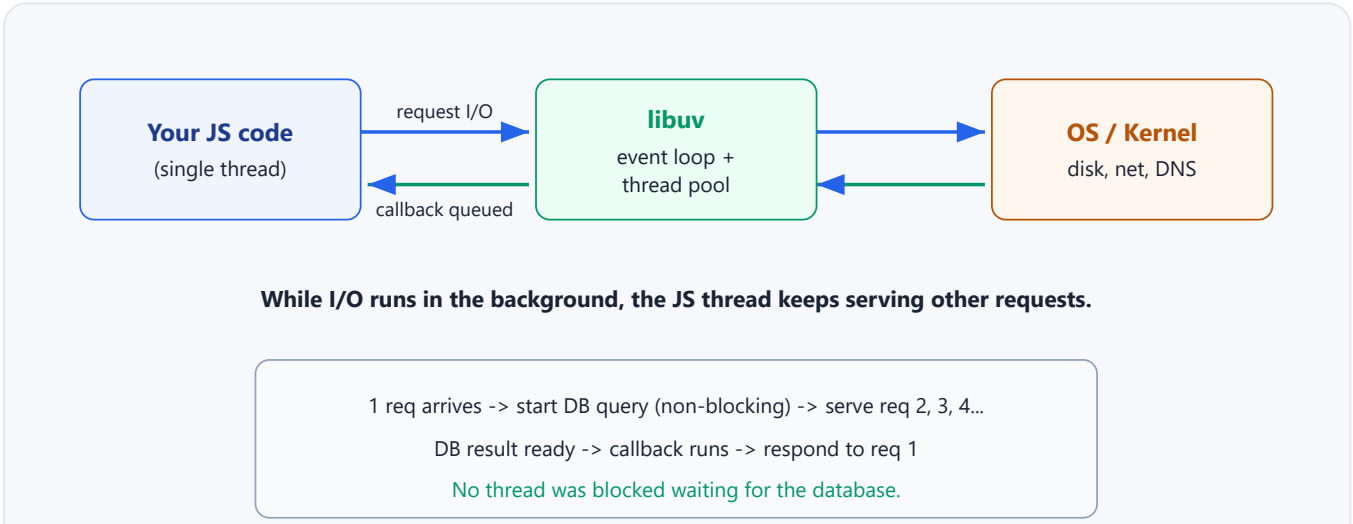


Fig 1. Node offloads slow I/O and keeps the single JS thread busy with other work.

GREAT FOR

I/O-bound workloads: REST/GraphQL APIs, real-time (WebSockets), BFFs, microservices, CLI tooling, SSR frontends.

WEAK FOR

CPU-bound work (video encode, big crypto, heavy math) blocks the single thread — offload to worker threads or another service.

SHIPS WITH

npm, a rich standard library, a native test runner, a REPL, and increasingly stable experimental features (fetch, watch mode).

> **Mental model**
> JavaScript itself is single-threaded, but Node is not. libuv maintains a thread pool (default 4 threads) for filesystem and some crypto/DNS work. "Single-threaded" refers to *your* JavaScript executing one callback at a time — never two in parallel.

2. Setup and Getting Started

Install Node via the official installer, a package manager, or a version manager. **nvm** (Node Version Manager) is strongly recommended so you can switch versions per project.

BASH

```
# Install nvm (macOS/Linux), then a Node LTS release
nvm install --lts
nvm use --lts
node --version        # v22.x (LTS as of 2026)
npm --version

# Quick REPL
node
> 2 + 2
4

# Run a script
node app.js

# Watch mode: auto-restart on file change (stable since v22, no nodemon needed)
node --watch app.js

# Run a one-off expression
node -e "console.log(process.platform)"
```

Every project begins with a `package.json` manifest. Create one and define scripts:

BASH

```
mkdir my-api && cd my-api
npm init -y                # generate package.json with defaults
npm pkg set type="module"  # opt into ES modules
npm install express        # add a dependency
```

JSON

```json
{
  "name": "my-api",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js",
    "test": "node --test"
  },
  "dependencies": {
    "express": "^4.21.0"
  }
}
```

Tip

Commit `package-lock.json`. It pins the exact dependency tree so `npm ci` installs byte-identical versions in CI and production.

3. Modules: CommonJS vs ESM

Node supports two module systems. **CommonJS (CJS)** is the original (`require` / `module.exports`); **ES Modules (ESM)** is the standardized JavaScript syntax (`import` / `export`). Node decides which to use based on the file extension and the `"type"` field in `package.json`.

COMMONJS

Default for `.cjs` files and packages without `"type": "module"`. Synchronous `require`. Has `__dirname`, `__filename`, `module`, `exports` as globals.

ES MODULES

Used for `.mjs` or when `"type": "module"`. Asynchronous, statically analyzable `import`. Top-level `await` allowed. The modern default for new projects.

JAVASCRIPT

```
// --- CommonJS (math.cjs) ---
function add(a, b) { return a + b; }
module.exports = { add };

// consume it
const { add } = require("./math.cjs");
console.log(add(2, 3)); // 5
```

JAVASCRIPT

```
// --- ES Modules (math.js, with "type":"module") ---
export function add(a, b) { return a + b; }
export default function mul(a, b) { return a * b; }

// consume it — note the explicit .js extension (required in ESM)
import mul, { add } from "./math.js";
console.log(add(2, 3), mul(2, 3)); // 5 6

// ESM lacks __dirname; reconstruct it:
import { fileURLToPath } from "node:url";
import { dirname } from "node:path";
const __dirname = dirname(fileURLToPath(import.meta.url));
```

⚠ **Watch out**

You cannot `require()` an ESM module from CJS the old way (though `import()` works, and Node 22+ can synchronously require ESM graphs that have no top-level await). Core modules should be imported with the `node:` prefix — e.g. `import { readFile } from "node:fs/promises"` — which is unambiguous and faster to resolve.

Feature	CommonJS	ES Modules
Import syntax	`const x = require( ... )`	`import x from ...`
Export syntax	`module.exports =`	`export` / `export default`
Loading	Synchronous	Asynchronous
Top-level await	No	Yes
File extension in path	Optional	Required
`__dirname`	Built-in	Derive from `import.meta.url`

4. The Event Loop in Depth

The event loop is libuv's heartbeat. Each iteration ("tick") processes a set of ordered **phases**, draining a queue of callbacks in each. Understanding the order explains why some callbacks fire before others.

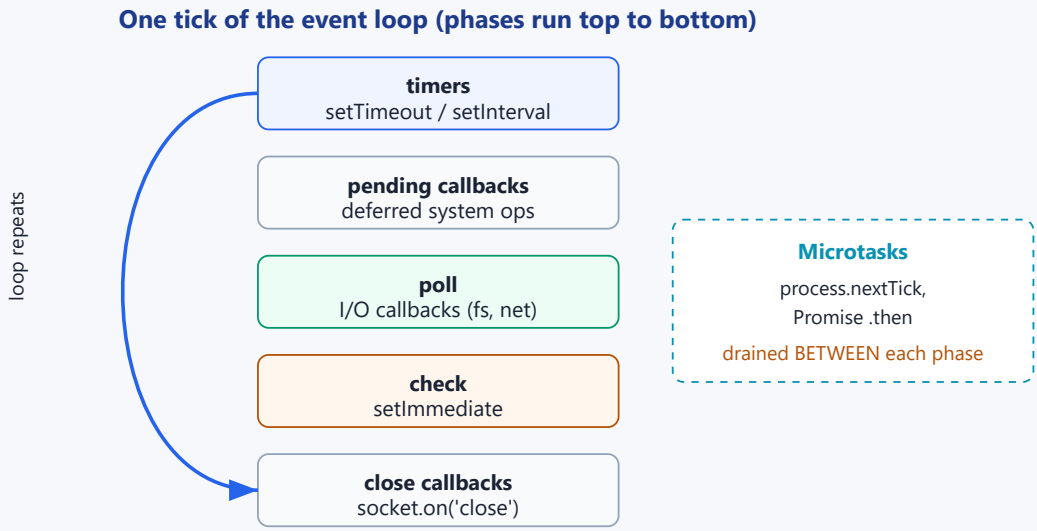


Fig 2. Event loop phases. Microtask queues (nextTick, promises) are emptied between every phase, not just once per tick.

The practical consequence: **microtasks beat macrotasks**. A resolved promise's `.then` runs before a `setTimeout(fn, 0)`, and `process.nextTick` runs before even promises.

JAVASCRIPT

```
console.log("1: sync start");

setTimeout(() => console.log("4: timeout (macrotask)"), 0);

Promise.resolve().then(() => console.log("3: promise (microtask)"));

process.nextTick(() => console.log("2: nextTick (runs first)"));

console.log("1b: sync end");

// Output order:
// 1: sync start
// 1b: sync end
// 2: nextTick (runs first)
// 3: promise (microtask)
// 4: timeout (macrotask)
```

✕ **Blocking is fatal**

A long synchronous loop or a huge `JSON.parse` freezes the entire server — no requests are served until it finishes. Never do heavy CPU work on the main thread; use `worker_threads`, streaming, or a separate service.

5. Async Patterns: Callbacks, Promises, async/await

Node's history is a progression of three async styles. Modern code almost always uses `async/await`, but you must recognize all three.

Callbacks (and callback hell)

JAVASCRIPT

```
import { readFile } from "node:fs";

// Node's classic "error-first callback" convention: (err, data) => {}
readFile("a.txt", "utf8", (err, a) => {
  if (err) return console.error(err);
  readFile("b.txt", "utf8", (err, b) => {   // nesting grows → "pyramid of doom"
    if (err) return console.error(err);
    console.log(a + b);
  });
});
```

Promises

JAVASCRIPT

```
import { readFile } from "node:fs/promises"; // promise-based API

readFile("a.txt", "utf8")
  .then(a => readFile("b.txt", "utf8").then(b => a + b))
  .then(combined => console.log(combined))
  .catch(err => console.error(err));
```

async/await (the modern default)

JAVASCRIPT

```
import { readFile } from "node:fs/promises";

async function concatFiles() {
  try {
    // Run independent I/O in PARALLEL with Promise.all — don't await serially
    const [a, b] = await Promise.all([
      readFile("a.txt", "utf8"),
      readFile("b.txt", "utf8"),
    ]);
    console.log(a + b);
  } catch (err) {
    console.error("Failed:", err.message);
  }
}
concatFiles();
```

✦ **Parallel vs serial**

`await a(); await b();` runs them one after another. If they don't depend on each other, start both first and `await Promise.all([a(), b()])` to overlap the waits. Use `Promise.allSettled` when you want every result even if some reject.

Combinator	Resolves when	Rejects when
`Promise.all`	All fulfill	Any rejects (fail-fast)
`Promise.allSettled`	All settle	Never (returns status array)
`Promise.race`	First settles	If first to settle rejects
`Promise.any`	First fulfills	All reject (AggregateError)

6. npm and the Package Ecosystem

npm is both a CLI and the world's largest software registry. It manages dependencies declared in `package.json` and installs them into `node_modules`.

Command	What it does
`npm install pkg`	Add runtime dependency, update lockfile
`npm install -D pkg`	Add devDependency (tooling, tests)
`npm ci`	Clean, reproducible install from lockfile (use in CI)
`npm run <script>`	Run a script defined in package.json
`npm outdated`	Show dependencies with newer versions
`npm audit fix`	Patch known security vulnerabilities
`npx <cmd>`	Run a package binary without installing globally

Semantic versioning (`MAJOR.MINOR.PATCH`) drives upgrade safety. Range specifiers in `package.json` control what `npm update` may bump to:



> **dependencies vs devDependencies**
> Runtime libraries (express, pg) go in `dependencies`. Build/test-only tools (eslint, vitest, typescript) go in `devDependencies` and are skipped by `npm ci --omit=dev` in production images.

7. Core Modules: Files, Paths, Process, Env

Node ships a large standard library. The ones you reach for daily:

JAVASCRIPT

```
import { readFile, writeFile, mkdir } from "node:fs/promises";
import { join, extname, basename } from "node:path";
import process from "node:process";

// Build cross-platform paths (never hand-concatenate with "/")
const dataDir = join(process.cwd(), "data");
await mkdir(dataDir, { recursive: true }); // no error if it already exists

const file = join(dataDir, "users.json");
await writeFile(file, JSON.stringify([{ id: 1 }], null, 2));

const raw = await readFile(file, "utf8");
console.log(JSON.parse(raw));
console.log(extname(file), basename(file)); // .json users.json
```

Environment variables & configuration

Read config from `process.env`. Node 20.6+ can load a `.env` file natively — no `dotenv` package needed.

BASH

```
# .env
PORT=3000
NODE_ENV=production
DATABASE_URL=postgres://localhost/mydb

# load it natively
node --env-file=.env server.js
```

JAVASCRIPT

```
// Validate and provide defaults — fail fast on missing required config
const PORT = Number(process.env.PORT ?? 3000);
const DB_URL = process.env.DATABASE_URL;
if (!DB_URL) {
  console.error("DATABASE_URL is required");
  process.exit(1); // non-zero exit signals failure to orchestrators
}

// Graceful shutdown: close connections before the process dies
process.on("SIGTERM", async () => {
  console.log("Shutting down...");
  // await server.close(); await db.end();
  process.exit(0);
});
```

Never commit secrets

Add `.env` to `.gitignore`. Secrets belong in your deployment platform's secret manager, injected as environment variables at runtime.

Streams & Buffers (handling large data)

Reading a 2 GB file into memory crashes. Streams process data in chunks with constant memory. A `Buffer` is Node's fixed-length raw byte container.

JAVASCRIPT

```
import { createReadStream, createWriteStream } from "node:fs";
import { pipeline } from "node:stream/promises";
import { createGzip } from "node:zlib";

// Stream + compress a huge file with backpressure handled for you
await pipeline(
  createReadStream("big.log"),
  createGzip(),
  createWriteStream("big.log.gz")
);
console.log("Compressed without loading the whole file into RAM.");
```

8. Building an HTTP Server

You can build a server with only the core `http` module — useful to understand what frameworks abstract away.

JAVASCRIPT

```
import { createServer } from "node:http";

const server = createServer((req, res) => {
  // req: IncomingMessage (method, url, headers)  res: ServerResponse
  if (req.method === "GET" && req.url === "/health") {
    res.writeHead(200, { "Content-Type": "application/json" });
    return res.end(JSON.stringify({ status: "ok" }));
  }
  res.writeHead(404, { "Content-Type": "text/plain" });
  res.end("Not Found");
});

server.listen(3000, () => console.log("Listening on http://localhost:3000"));
```

Raw `http` gets verbose fast: manual routing, body parsing, and header management. That's why nearly everyone reaches for a framework. **Express** remains the most widely used; **Fastify** is a faster modern alternative with built-in schema validation. We'll use Express.

9. Express: Routing, Middleware, and REST

Express models a request as flowing through an ordered **middleware pipeline**. Each middleware is a function `(req, res, next)`. It can read/modify `req` / `res`, end the response, or call `next()` to pass control onward.

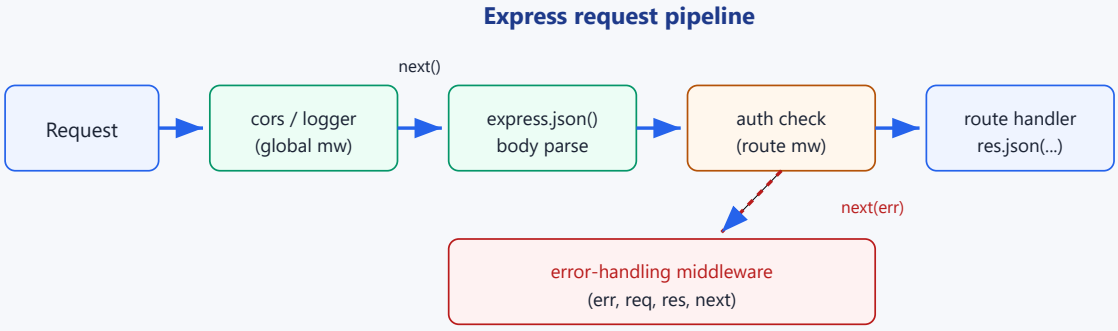


Fig 3. Middleware runs in registration order. Calling next(err) skips ahead to the error handler.

JAVASCRIPT

```
import express from "express";

const app = express();

// --- Global middleware (runs for every request, in order) ---
app.use(express.json());                  // parse JSON bodies into req.body
app.use((req, res, next) => {             // custom logger
  console.log(`${req.method} ${req.path}`);
  next();                                 // MUST call next() or the request hangs
});

// --- Routing: method + path → handler ---
app.get("/users/:id", (req, res) => {
  const { id } = req.params;              // route params  → /users/42
  const { verbose } = req.query;          // query string  → ?verbose=true
  res.json({ id, verbose: verbose === "true" });
});

app.post("/users", (req, res) => {
  const user = req.body;                  // parsed by express.json()
  res.status(201).json({ id: 99, ...user });
});

app.listen(3000, () => console.log("API on :3000"));
```

Routers for modular structure

JAVASCRIPT

```
// routes/users.js — a mountable sub-application
import { Router } from "express";
const router = Router();

router.get("/", (req, res) => res.json([{ id: 1 }]));
router.get("/:id", (req, res) => res.json({ id: req.params.id }));
export default router;

// server.js
import usersRouter from "./routes/users.js";
app.use("/api/users", usersRouter); // all routes now prefixed with /api/users
```

HTTP method	REST meaning	Typical status
GET	Read a resource / collection	200 OK
POST	Create a new resource	201 Created
PUT	Replace a resource fully	200 / 204
PATCH	Partially update	200
DELETE	Remove a resource	204 No Content

10. Error Handling

Errors in async code are the #1 source of production crashes. Rules of thumb:

IN ASYNC/AWAIT

Wrap awaits in `try/catch`. An unhandled rejection in a route handler won't be caught by Express automatically (in v4) — forward it with `next(err)`.

CENTRALIZE

Define ONE error-handling middleware with four args `(err, req, res, next)` at the end. Translate error types into HTTP responses there.

JAVASCRIPT

```
// A typed application error
class AppError extends Error {
  constructor(message, status = 500) {
    super(message);
    this.status = status;
  }
}

// Wrapper so async handler rejections reach the error middleware
const wrap = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(nex

app.get("/users/:id", wrap(async (req, res) => {
  const user = await db.find(req.params.id);
  if (!user) throw new AppError("User not found", 404);
  res.json(user);
}));

// 404 for unmatched routes (register AFTER all routes)
app.use((req, res) => res.status(404).json({ error: "Not Found" }));

// Central error handler — note the FOUR parameters
app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status ≥ 500) console.error(err); // log server faults, not client ones
  res.status(status).json({ error: err.message });
});
```

⚠ **Process-level safety nets**

Attach `process.on("unhandledRejection")` and `process.on("uncaughtException")` to log and exit gracefully. These are last resorts for logging, not a substitute for local `try/catch`. After an uncaught exception the process is in an undefined state — log it and let it restart.

✦ **Express 5**

In Express 5 (stable 2024+), rejected promises returned from async route handlers are automatically forwarded to the error handler, so the `wrap` helper is no longer needed. On v4 it still is.

11. Worked Mini-Example: A REST API Backend

Let's assemble the pieces into a small but complete "tasks" API with in-memory storage, validation, and centralized errors. This is the shape of a real backend minus the database.

JAVASCRIPT

```
// server.js  -  run with:  node --watch --env-file=.env server.js
import express from "express";

const app = express();
app.use(express.json());

// --- "Database" (swap for Postgres/Mongo in production) ---
const tasks = new Map();
let nextId = 1;

class AppError extends Error {
  constructor(msg, status = 500) { super(msg); this.status = status; }
}
const wrap = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// --- CREATE ---
app.post("/tasks", wrap(async (req, res) => {
  const { title, done = false } = req.body ?? {};
  if (typeof title !== "string" || title.trim() === "") {
    throw new AppError("title is required", 400);
  }
  const task = { id: nextId++, title: title.trim(), done: Boolean(done) };
  tasks.set(task.id, task);
  res.status(201).json(task);
}));

// --- READ collection (with simple filtering) ---
app.get("/tasks", wrap(async (req, res) => {
  let list = [...tasks.values()];
  if (req.query.done !== undefined) {
    const want = req.query.done === "true";
    list = list.filter(t => t.done === want);
  }
  res.json(list);
}));

// --- READ one ---
app.get("/tasks/:id", wrap(async (req, res) => {
  const task = tasks.get(Number(req.params.id));
  if (!task) throw new AppError("task not found", 404);
  res.json(task);
}));

// --- UPDATE (partial) ---
app.patch("/tasks/:id", wrap(async (req, res) => {
  const task = tasks.get(Number(req.params.id));
  if (!task) throw new AppError("task not found", 404);
```

```
  const { title, done } = req.body ?? {};
  if (title !== undefined) task.title = String(title);
  if (done !== undefined) task.done = Boolean(done);
  res.json(task);
}));

// --- DELETE ---
app.delete("/tasks/:id", wrap(async (req, res) => {
  const existed = tasks.delete(Number(req.params.id));
  if (!existed) throw new AppError("task not found", 404);
  res.status(204).end();
}));

// --- 404 + central error handler (order matters) ---
app.use((req, res) => res.status(404).json({ error: "Not Found" }));
app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) console.error(err);
  res.status(status).json({ error: err.message });
});

const PORT = Number(process.env.PORT ?? 3000);
app.listen(PORT, () => console.log(`Tasks API on :${PORT}`));
```

BASH

```
# Exercise it with curl
curl -X POST localhost:3000/tasks -H 'Content-Type: application/json' \
     -d '{"title":"Learn Node"}'
# {"id":1,"title":"Learn Node","done":false}

curl localhost:3000/tasks
curl -X PATCH localhost:3000/tasks/1 -H 'Content-Type: application/json' \
     -d '{"done":true}'
curl -X DELETE localhost:3000/tasks/1 -w "%{http_code}\n"   # 204
```

Show a minimal test using the built-in test runner

12. Cheat-Sheet Reference

Task	API
Read file (async)	`await readFile(path, "utf8")` from `node:fs/promises`
Join paths	`join(dir, "file.txt")` from `node:path`
Env var	`process.env.NAME`
CLI args	`process.argv.slice(2)`
Exit with error	`process.exit(1)`
Current dir	`process.cwd()`
Run parallel async	`await Promise.all([ ... ])`
Defer to next tick	`queueMicrotask(fn)` / `setImmediate(fn)`
HTTP request (built-in)	`await fetch(url)` (global since v18)
Parse JSON body (Express)	`app.use(express.json())`
Set status + JSON	`res.status(201).json(obj)`
Route param	`req.params.id`

Runtime comparison	Node.js	Deno	Bun
Engine	V8	V8	JavaScriptCore
TypeScript	Native strip (v22+)	First-class	First-class
Package manager	npm	npm / JSR	bun (npm-compatible)
Maturity / ecosystem	Largest, battle-tested	Growing	Fast, newer

13. Best Practices and Common Pitfalls

✓ **Do**

Use `async/await` with `try/catch` and a central error handler. Validate all external input (bodies, params, env). Prefer `node:fs/promises` over callback APIs. Pin dependencies with a committed lockfile and run `npm ci` in CI.

✓ **Do**

Keep handlers thin — push logic into a service layer. Stream large payloads. Set security middleware (helmet, rate limiting, CORS) and read config from env. Implement graceful shutdown on `SIGTERM`.

✦ **Tip**

Overlap independent I/O with `Promise.all`. Use the `node:` prefix for core modules. Add structured logging (pino) rather than scattering `console.log`.

⚠ **Pitfall**

Forgetting `next()` in middleware hangs the request forever. Mixing `await` inside a `forEach` does NOT wait — use a `for ... of` loop or `Promise.all(array.map( ... ))`. Floating promises (no `await`, no `.catch`) swallow errors.

✕ **Danger**

Blocking the event loop with CPU-heavy sync code stalls every connection. Never store secrets in code or logs. Don't trust `req.body` without validation — it is attacker-controlled. A single uncaught exception can leave the process in a corrupt state; restart rather than continue.

14. Interview Questions & Answers

> **Is Node.js single-threaded?**
>
> Your JavaScript executes on a single main thread, so only one callback runs at a time. But Node is not single-threaded overall: libuv maintains a thread pool (default 4) for filesystem/DNS/some

crypto work, and the kernel handles network I/O asynchronously. You can also spawn `worker_threads` for CPU-bound parallelism.

Explain the event loop phases.

Each tick runs ordered phases: timers (setTimeout/setInterval), pending callbacks, poll (I/O callbacks), check (setImmediate), and close callbacks. Between every phase, the microtask queues are fully drained — first `process.nextTick`, then resolved promises. That's why a promise callback runs before a `setTimeout(0)`.

What's the difference between `process.nextTick` and `setImmediate`?

`process.nextTick` callbacks run immediately after the current operation, before the event loop continues and before promise microtasks — so a tight recursive `nextTick` can starve the loop. `setImmediate` runs in the "check" phase, after the current poll phase completes. Use `setImmediate` to yield to I/O; use `nextTick` sparingly.

CommonJS vs ES Modules — key differences?

CJS uses synchronous `require` / `module.exports` and has `__dirname`; ESM uses asynchronous, statically analyzable `import` / `export`, requires file extensions, allows top-level await, and derives `__dirname` from `import.meta.url`. The system is chosen by file extension (`.cjs` / `.mjs`) or the `"type"` field in package.json.

How do you run multiple async operations efficiently?

If they're independent, start them all and await together: `await Promise.all([a(), b()])` overlaps the waits. Awaiting serially (`await a(); await b();`) is slower because each waits for the previous. Use `Promise.allSettled` when you need every result even if some fail, and `Promise.any` for the first success.

What is middleware in Express and what order does it run?

Middleware is a function `(req, res, next)` that runs in the order registered. It can inspect/modify the request/response, end the response, or call `next()` to pass control. Error-handling middleware has four parameters `(err, req, res, next)` and is triggered by `next(err)`; it must be registered last.

How do you handle errors in async Express routes?

In Express 4, a rejected promise in an async handler isn't caught automatically, so you wrap the handler to `.catch(next)`, forwarding to a central error middleware. Express 5 does this automatically. Either way, centralize error-to-HTTP mapping in one error handler, and add process-level `unhandledRejection` / `uncaughtException` listeners as logging safety nets.

What happens if you block the event loop?

A long synchronous operation (big loop, synchronous crypto, giant JSON.parse) stops the single JS thread, so no other callbacks — including incoming requests — run until it finishes. The server appears frozen. Fix by streaming, chunking work, offloading to `worker_threads`, or moving the computation to another service.

What's the difference between `dependencies` and `devDependencies`?

`dependencies` are needed at runtime (express, pg) and are installed in production. `devDependencies` are only for development and build (eslint, test runners, typescript) and are omitted with `npm ci --omit=dev`. Keeping them separate shrinks production images and attack surface.

Why use streams instead of reading a whole file?

Streams process data in chunks with roughly constant memory, so you can handle files or network payloads larger than available RAM, and you start producing output before all input arrives. `pipeline()` wires streams together and handles backpressure and cleanup automatically.

How does `require` caching work?

The first `require` of a module executes it and caches the resulting `module.exports` in `require.cache` keyed by resolved path. Subsequent requires return the same cached object without re-executing — so modules are effectively singletons. ESM has analogous one-time evaluation of each module graph node.

How would you scale a Node app across CPU cores?

Because one process uses one core for JS, run multiple processes: the built-in `cluster` module or a process manager (PM2), or horizontal scaling behind a load balancer / orchestrator (Kubernetes). Keep the app stateless (store sessions in Redis) so any instance can serve any request. For CPU hotspots within a process, use `worker_threads`.

15. Further Learning

- **Official docs:** nodejs.org/docs and the Node.js Learn guides — authoritative and current.
- **Deepen the event loop:** read the libuv design overview and experiment with `perf_hooks` to measure lag.
- **Frameworks:** compare Express with Fastify (schema validation, speed) and NestJS (opinionated, TypeScript, DI).
- **Data layer:** add a real database with Prisma or Drizzle ORM, or `pg` / `mongodb` drivers directly.
- **Production hardening:** helmet, rate limiting, input validation with zod, structured logging with pino, OpenTelemetry tracing.

- **TypeScript:** adopt it for safety; Node 22+ can run `.ts` via native type stripping, or use `tsx` / `ts-node`.
- **Testing:** the built-in `node:test` runner, or Vitest; use supertest for HTTP-level route tests.
- **Build something:** extend the tasks API with a database, auth (JWT), pagination, and deploy it to a container platform.